

osmnx-sandbox

November 16, 2025

```
[1]: %reload_ext autoreload  
      %autoreload 2
```

```
[2]: from kret_studies import *  
      from kret_studies.notebook import *  
      from kret_studies.complex import *  
  
      logger = get_notebook_logger()
```

Loaded environment variables from /Users/Akseldkw/coding/Columbia/RL-Project/.env.

INFO:datasets:JAX version 0.7.2 available.

```
[3]: from waymo_agent import *
```

```
[4]: G = ox.load_graphml(filepath=MAP_DIR / "manhattan.graphml")
```

```
[5]: fig, ax = ox.plot_graph(G, node_size=0, edge_linewidth=0.5)
```



```
[6]: # 1. Load graph (if not already loaded)
# G = ox.load_graphml("manhattan.graphml")

# 2. Define mappings for Colors AND Widths
config = {
    "motorway": {"color": "#ff0000", "width": 3.0}, # Red, Thickest
    "motorway_link": {"color": "#ff0000", "width": 3.0},
    "trunk": {"color": "#ff6600", "width": 2.5}, # Orange, Thick
    "primary": {"color": "#ffac12", "width": 2.0}, # Gold, Medium-Thick
    "secondary": {"color": "#f4f458", "width": 1.0}, # Yellow, Medium
```

```

    "tertiary": {"color": "#ffffff", "width": 0.8}, # White, Thin
    "residential": {"color": "#555555", "width": 0.5}, # Grey, Thinnest
    "default": {"color": "#333333", "width": 0.5}, # Fallback
}

# 3. Generate the lists
edge_colors = []
edge_widths = []

for u, v, data in G.edges(data=True):
    # Get highway type (handle lists if necessary)
    highway_type = data.get("highway", "default")
    if isinstance(highway_type, list):
        highway_type = highway_type[0]

    # Fetch style from config, or use default if type not found
    style = config.get(highway_type, config["default"])

    edge_colors.append(style["color"])
    edge_widths.append(style["width"])

# 4. Plot with both attributes
fig, ax = ox.plot_graph(
    G,
    node_size=0,
    edge_color=edge_colors,
    edge_linewidth=edge_widths, # <--- Pass the new list here
    bgcolor="black",
    figsize=(12, 12), # Bigger size helps see the width differences
    show=True,
)

```



```

[7]: fig.savefig(IMG_DIR / "manhattan_map.png", dpi=600, bbox_inches="tight",
    ↪facecolor=fig.get_facecolor())

[8]: len(G.nodes), len(G.edges)

[8]: (4619, 9901)

[9]: type(G)

[9]: networkx.classes.multidigraph.MultiDiGraph

[10]: G_full = G
    G.graph.get("simplified")

[10]: True

[11]: G_sparse = sparsify_graph(G_full)
    print(f"Sparsified: {G_sparse.number_of_nodes()} nodes, {G_sparse.
    ↪number_of_edges()} edges")

    # --- --- --- --- ---
    # PLOT THE COMPARISON
    # --- --- --- --- ---
    print("Plotting comparison...")

    # Define plot styles
    plot_kwargs = {"node_size": 0, "edge_linewidth": 0.8, "bgcolor": "black"}
    major_color = "#ffac12" # Gold for major roads

    # Create a 1x2 subplot (one row, two columns)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))

    # Plot 1: Full Graph (colored from our previous step)
    # (Using simplified colors for this example)
    ec_full = ["#333333" if data.get("highway") == "residential" else major_color,
    ↪for u, v, data in G_full.edges(data=True)]

    ox.plot_graph(G_full, ax=ax1, edge_color=ec_full, **plot_kwargs)
    ax1.set_title(f"Full Graph ({G_full.number_of_nodes()} nodes)", color="white")

    # Plot 2: Sparsified Graph
    ox.plot_graph(G_sparse, ax=ax2, edge_color=major_color, **plot_kwargs)

```

```

ax2.set_title(f"Sparsified Graph ({G_sparse.number_of_nodes()} nodes)",
    ↪color="white")

fig.set_facecolor("black")
plt.suptitle("Manhattan Road Network Sparsification", fontsize=20,
    ↪color="white")
plt.tight_layout()

# Save and show
fig.savefig(IMG_DIR / "manhattan-sparse.png", dpi=300, bbox_inches="tight",
    ↪facecolor="black")
plt.show()

```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[11], line 1
----> 1 G_sparse = sparsify_graph(G_full)
      2 print(f"Sparsified: {G_sparse.number_of_nodes()} nodes, {G_sparse.
    ↪number_of_edges()} edges")
      5 # --- --- --- --- ---
      6 # PLOT THE COMPARISON
      7 # --- --- --- --- ---

File ~/coding/Columbia/RL-Project/waymo_agent/graph/sparsify.py:67, in
    ↪sparsify_graph(G)
      61 # 4. *** NEW STEP: Simplify Topology ***
      62 # This is the step you requested. It removes all "redundant" nodes
      63 # (nodes that are not true intersections) and merges the edges on
      64 # either side, but only if their attributes (name, oneway, etc.) match.
      65 # This cleans up the graph after we removed all the minor roads.
      66 assert isinstance(G_sparsified, nx.MultiDiGraph)
----> 67 G_simplified = ox.simplify_graph(G_sparsified)
      69 assert isinstance(G_simplified, nx.MultiDiGraph)
      71 return G_simplified

File ~/micromamba/envs/kret_312/lib/python3.12/site-packages/osmnx/
    ↪simplification.py:406, in simplify_graph(G, node_attrs_include,
    ↪edge_attrs_differ, remove_rings, track_merged, edge_attr_aggs)
      404     agg_func = edge_attr_aggs[attr_name]
      405     path_attributes[attr_name] = agg_func(attr_values)
--> 406 elif len(set(attr_values)) == 1:
      407     # if there's only 1 unique value, keep that single value
      408     path_attributes[attr_name] = attr_values[0]
      409 else:
      410     # otherwise, if there are multiple uniques, keep one of each

```

```
TypeError: unhashable type: 'list'
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```